Lecture 15: Routing — Link State and Dijkstra's Algorithm

How routers build forwarding tables from a complete network map

EC 441 — Introduction to Computer Networking
Boston University

Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

- Represent a network as a **weighted graph**: nodes, edges, weights, directionality
- Explain how **link-state routing** works: flooding gives every router the full topology
- Execute **Dijkstra's algorithm** by hand on a small network graph
- Construct a **shortest-path tree** and derive a **forwarding table** from it
- Describe how **OSPF** implements link-state routing in practice
- Identify the **oscillation problem** with traffic-sensitive link costs
- Use Python's `networkx` to build graphs, run Dijkstra, and visualize paths

Key Acronyms

Acronym	Meaning
ABR	Area Border Router
BFS	Breadth-First Search
ECMP	Equal-Cost Multi-Path
IGP	Interior Gateway Protocol
IS-IS	Intermediate System to IS
LSA	Link-State Advertisement

Acronym	Meaning
LSDB	Link-State Database
MTR	My TraceRoute
OSPF	Open Shortest Path First
SPF	Shortest Path First
SPT	Shortest-Path Tree
TTL	Time To Live

The Question We Haven't Answered

L13: every router has a **forwarding table** (destination prefix → output link). **L14**: those prefixes are CIDR blocks from hierarchical address allocation.

The remaining question

How does the forwarding table get built?

Static routing: a human configures every entry. Does not scale. **Dynamic routing**: routers **automatically** compute forwarding tables and **adapt** when the network changes.

Two major families:

1. **Link-state** (today) — every router learns the full topology, computes shortest paths
2. **Distance-vector** (L16) — routers learn only from neighbors, converge iteratively

A useful duality

Link-state: *local* information distributed *globally* — each router shares its own links with everyone.

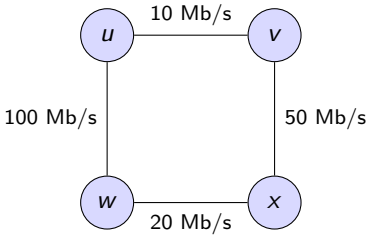
Distance-vector: *global* information distributed *locally* — each router shares its distances to all destinations with only its neighbors.

The Graph Abstraction

A computer network maps directly to a **graph** $G = (V, E)$:

- V = set of **vertices** (also called **nodes**)
- E = set of **edges** (also called **links**)
- In networking: vertices = routers, edges = links between them

Graph term	Networking term
Vertex / node	Router
Edge / link	Physical or logical connection
Adjacent	Directly connected
Degree	Number of links on a router
Path	Sequence of hops



Four routers, four links

$|V| = 4$, $|E| = 4$

[Wikipedia: Graph (discrete mathematics)]

Edge Weights (Costs)

A **weighted graph** assigns a numerical **cost** to each edge. What the cost represents determines what "shortest path" means.

Weight meaning	Lower is...	Used by
Hop count	Fewer routers traversed	RIP (all costs = 1)
Inverse bandwidth	Higher-capacity link	OSPF default: $10^8/B$
Propagation delay	Lower latency	Delay-sensitive routing
Administrative cost	Preferred by policy	Operator-configured

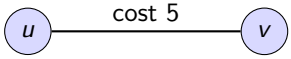
OSPF default cost: cost = 10^8 / B = transmission time (ms) of 100 kb $\approx$ 8 packets

Connects to $d_{\text{trans}} = L/R$ — OSPF cost **is** transmission delay. Summing costs along a path = summing tx delays.

Link type	Bandwidth	OSPF cost	tx time of 100 kb
T1	1.544 Mb/s	64	64 ms
10 Mb/s Ethernet	10 Mb/s	10	10 ms
100 Mb/s Fast Ethernet	100 Mb/s	1	1 ms

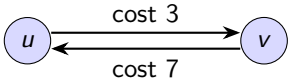
Directed vs. Undirected Edges

Undirected edge
Symmetric — same cost both ways. Most full-duplex links.



Cost $u \to v$ = cost $v \to u = 5$

Directed edges (arcs)
Asymmetric — different cost per direction.



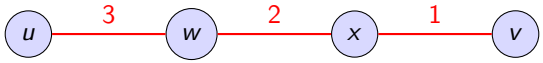
E.g., satellite: uplink (slow) vs. downlink (fast)

Convention for this course

We primarily use **undirected graphs** — most network links are symmetric. When directionality matters, we will note it explicitly.

Paths, Path Cost, and Shortest Paths

A **path** from u to v is a sequence of vertices connected by edges.



$$\text{cost}(u \to w \to x \to v) = c(u, w) + c(w, x) + c(x, v) = 3 + 2 + 1 = 6$$

A **shortest path** from u to v is a path with minimum total cost.

Key definitions:

- **Adjacent** (neighbors) = share an edge
- **Degree** = number of edges on a node
- **Cycle** = path that starts and ends at the same vertex

Routing = shortest paths

The routing problem is: find the shortest (minimum-cost) path from each router to every destination.

Trees and Shortest-Path Trees

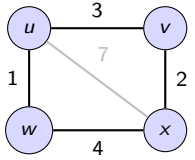
A **tree** = connected graph with no cycles.
A **spanning tree** of G = subgraph that includes all vertices, with exactly $|V| - 1$ edges (no cycles).

A **shortest-path tree (SPT)** rooted at u = spanning tree where the path from u to every other vertex is a shortest path in the original graph.

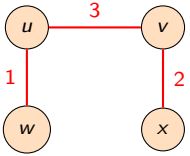
The key insight

Dijkstra's algorithm computes an SPT. The SPT is exactly what a router needs to build its forwarding table.

Original graph (has cycles):



SPT rooted at u:



3 edges, 4 nodes, no cycles

Demo: traceroute

Demo: see the graph in action — each hop is a vertex, each jump is an edge

```
$ traceroute google.com
 1 192.168.1.1 1.2 ms "home router" (vertex 1)
 2 10.0.0.1 5.4 ms "ISP router" (vertex 2)
 3 72.14.215.85 8.8 ms "Google edge" (vertex 3)
 4 142.250.80.46 9.1 ms "destination" (vertex 4)
```

The routing algorithm determined this specific path through the network graph. Different source locations would show different paths.

mtr — My TraceRoute (Linux)

mtr combines traceroute with continuous ping, showing real-time per-hop loss and latency statistics. Available on Linux natively; on macOS via Homebrew (brew install mtr), but requires sudo for raw socket access.

Link-State Routing: The Big Idea

Link-state approach:
Every router gets a **complete map**.
Each computes shortest paths **independently**.

Analogy:
Having a **full road map** and computing your own best route.

Distance-vector approach (L16):
Each router knows only its **direct neighbors** and their distance estimates.

Analogy:
Asking **locals for directions** at each intersection.

The link-state process has three phases:

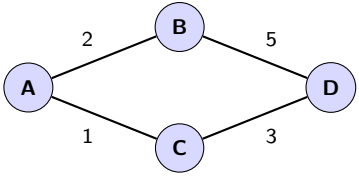
After flooding: every router has the **same topology database**.
Dijkstra is deterministic $\Rightarrow$ all routers compute **consistent** forwarding tables.

[Wikipedia: Link-state routing protocol]

Phase 1–2: Discover and Flood

Phase 1: Each router sends **hello packets** on all interfaces, discovers neighbors and link costs. Records this as a **Link-State Advertisement (LSA)**.

Phase 2: Each router **floods** its LSA to all other routers:



After flooding, every router knows:

A's LSA: {A–B: 2, A–C: 1}
B's LSA: {B–A: 2, B–D: 5}
C's LSA: {C–A: 1, C–D: 3}
D's LSA: {D–B: 5, D–C: 3}

Same data at every router.

Flooding rules (prevent infinite loops):

- Each LSA has a **sequence number** — routers discard already-seen LSAs
- Forward received LSA out **all interfaces except the one it arrived on**
- LSAs have a **maximum age** — refreshed periodically (every 30 min in OSPF)

Phase 3: Dijkstra's Algorithm

Given: weighted graph $G = (V, E)$ and source node s.
Find: shortest path from s to every other node.

Invented by Edsger Dijkstra (1956, published 1959). One of the most important algorithms in CS.

The algorithm maintains:

- $D(v)$ — current best-known cost from s to v
- $p(v)$ — predecessor of v on the best-known path
- N' — the set of nodes whose shortest path is **finalized**

Key property

When a node w is added to N', its cost $D(w)$ is **guaranteed** to be the true shortest-path cost. Why? All edge weights are non-negative, so any other path through un-finalized nodes cannot be shorter.

[Wikipedia: Dijkstra's algorithm]

Dijkstra's Algorithm: Pseudocode

```
Initialize:
  N' = {s} // start with source only
  D(s) = 0 // cost to yourself = 0
  For all neighbors v of s:
    D(v) = c(s,v) // direct link cost
    p(v) = s // predecessor = source
  For all other nodes v:
    D(v) = ∞ // not yet reachable

Loop (until N' contains all nodes):
  Find w ∉ N' with minimum D(w) // closest un-finalized node
  Add w to N' // w is now finalized
  For each neighbor v of w where v ∉ N':
    If D(w) + c(w,v) < D(v): // shorter path through w?
      D(v) = D(w) + c(w,v) // update cost
      p(v) = w // update predecessor
```

Complexity: $O(|V|^2)$ with array; $O((|V| + |E|) \log |V|)$ with priority queue. **Requires non-negative edge weights** (Bellman-Ford handles negatives — L16).

Relaxation and Why Dijkstra Is Optimal

Relaxation — testing whether a path through u improves the cost to v:

$$\text{If } D(u) + c(u, v) < D(v), \quad \text{then set } D(v) = D(u) + c(u, v)$$

(The name: $D(v)$ is "too tight" and we "relax" it to a better value.)

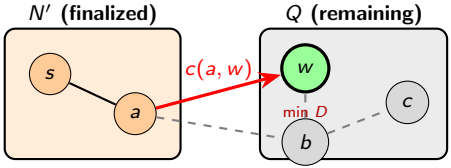
Why is the greedy choice correct?

When we pick w with **min** $D(w)$ from Q:

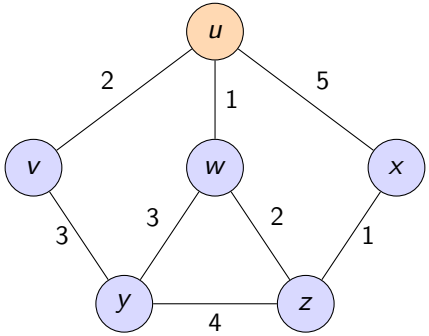
- Any other path to w crosses the cut through some $b \in Q$
- $D(b) \geq D(w)$ (we chose min), remaining edges ≥ 0
- No other path can be shorter

$\Rightarrow D(w)$ is the **true shortest-path cost**.

This is why negative weights break Dijkstra.



Worked Example: The Network



Source node: u (orange). Six nodes, eight edges.

Edge	u–v	u–w	u–x	v–y	w–y	w–z	x–z
Cost	2	1	5	3	3	2	1

Edge	y–z
Cost	4

Dijkstra Step-by-Step (1/3)

Initialization: $N' = \{u\}$, set costs to direct neighbors, ∞ for rest.

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$	Select
Init	$\{u\}$	2, u	1, u	5, u	∞	∞	—

Step 1: Minimum is w ($D = 1$). Add w to N'. Update w's neighbors:

- y: $D(w) + c(w, y) = 1 + 3 = 4 < \infty \Rightarrow D(y) = 4$, $p(y) = w$
- z: $D(w) + c(w, z) = 1 + 2 = 3 < \infty \Rightarrow D(z) = 3$, $p(z) = w$

Step	N'	$D(v)$	$D(w)$	$D(x)$	$D(y)$	$D(z)$	Select
Init	$\{u\}$	2	1	5	∞	∞	—
1	$\{u, w\}$	2	**1**	5	4	3	w

Dijkstra Step-by-Step (2/3)

Step 2: Minimum is v ($D = 2$). Add v to N'. Update:

- y: $D(v) + c(v, y) = 2 + 3 = 5 > 4 \Rightarrow$ no change (path through w is better)

Step 3: Minimum is z ($D = 3$). Add z to N'. Update:

- x: $D(z) + c(z, x) = 3 + 1 = 4 < 5 \Rightarrow D(x) = 4$, $p(x) = z$ **improvement!**
- y: $D(z) + c(z, y) = 3 + 4 = 7 > 4 \Rightarrow$ no change

Step	N'	$D(v)$	$D(w)$	$D(x)$	$D(y)$	$D(z)$	Select
Init	$\{u\}$	2	1	5	∞	∞	—
1	$\{u, w\}$	2	**1**	5	4	3	w
2	$\{u, w, v\}$	**2**	—	5	4	3	v
3	$\{u, w, v, z\}$	—	—	$5 \rightarrow 4$	4	**3**	z

Step 3 discovered a better path to x: $u \rightarrow w \rightarrow z \rightarrow x$ (cost 4) beats the direct path $u \rightarrow x$ (cost 5).

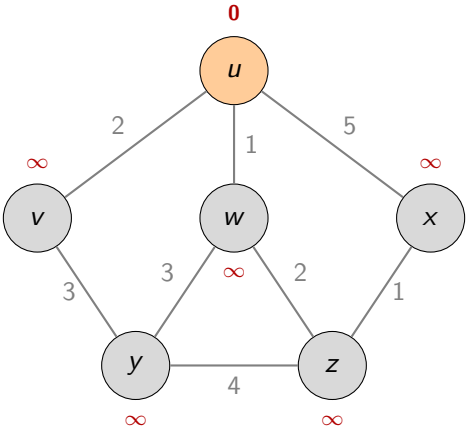
Dijkstra Step-by-Step (3/3)

Step 4: Tie between x and y (both $D = 4$). Pick x. No improvements.
Step 5: y is the last node. $D(y) = 4$, $p(y) = w$. Done.

Step	N'	$D(v)$	$D(w)$	$D(x)$	$D(y)$	$D(z)$	Select
Init	$\{u\}$	2	1	5	∞	∞	—
1	$\{u, w\}$	2	**1**	5	4	3	w
2	$\{u, w, v\}$	**2**	—	5	4	3	v
3	$\{u, w, v, z\}$	—	—	$5 \to 4$	4	**3**	z
4	$\{u, w, v, z, x\}$	—	—	**4**	4	—	x
5	$\{u, w, v, z, x, y\}$	—	—	—	**4**	—	y

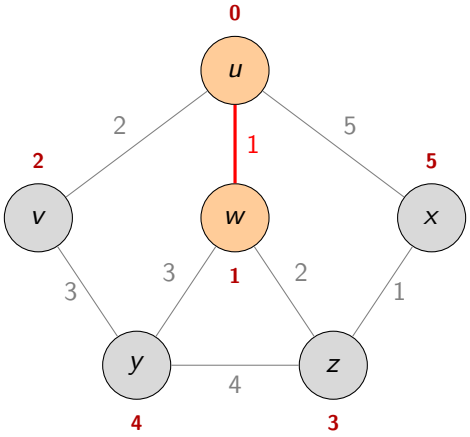
Final shortest-path costs from u: v: 2 w: 1 z: 3 x: 4 y: 4

Visual Walkthrough: Initialization



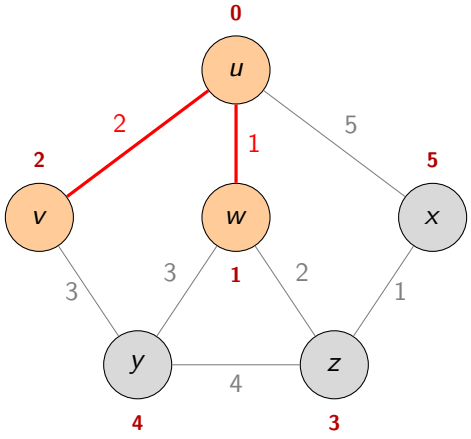
$N' = \{u\}$. Source node finalized with cost 0. All other nodes at ∞ — not yet reached.

Visual Walkthrough: Step 1 — Select w (cost 1)



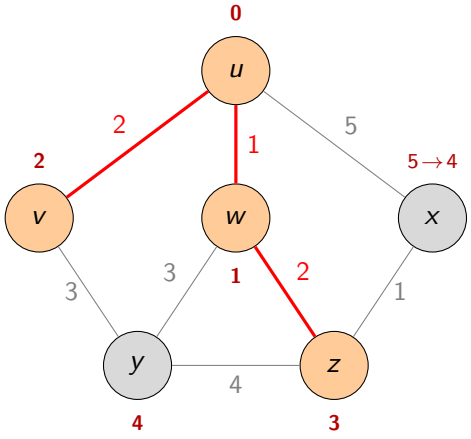
$N' = \{u, w\}$. w is closest ($D = 1$). Relaxing from w: discovered y (cost 4) and z (cost 3).

Visual Walkthrough: Step 2 — Select v (cost 2)



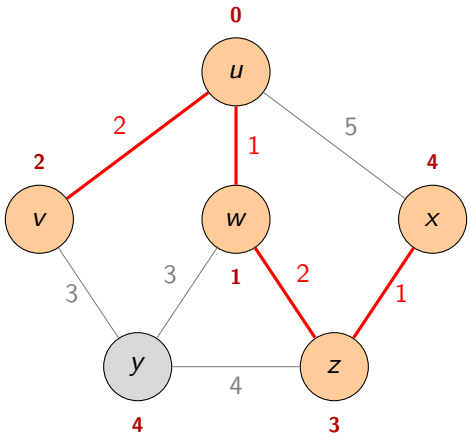
$N' = \{u, w, v\}$. v is next ($D = 2$). Checked y via v: $2 + 3 = 5 > 4$ — no improvement (path through w wins).

Visual Walkthrough: Step 3 — Select z (cost 3)



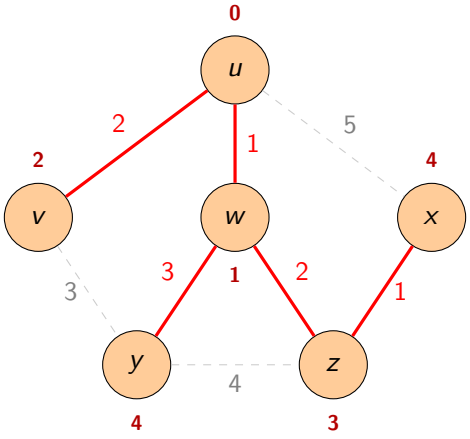
$N' = \{u, w, v, z\}$. z is next ($D = 3$). Key update: x improved from 5 to **4** via $u \to w \to z \to x$. The direct edge u–x is **not** on the shortest path!

Visual Walkthrough: Step 4 — Select x (cost 4)



$N' = \{u, w, v, z, x\}$. Tie between x and y (both $D = 4$), pick x. No improvements found. One node remains.

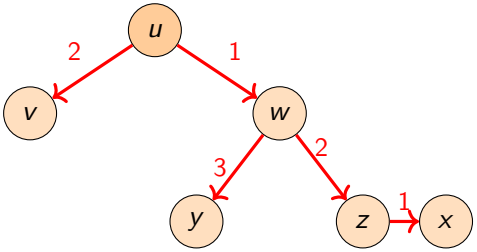
Visual Walkthrough: Step 5 — Select y (cost 4) — Done



$N' = \{u, w, v, z, x, y\}$. **All nodes finalized.** Red edges = shortest-path tree. Dashed gray = edges not in SPT. 5 tree edges, 3 unused — the SPT has $|V| - 1 = 5$ edges, no cycles.

The Shortest-Path Tree

From the predecessor pointers $p(v)$, reconstruct the **shortest-path tree** rooted at u:



5 edges, 6 nodes, no cycles

Shortest paths from u:

Dest	Path	Cost
v	$u \to v$	2
w	$u \to w$	1
y	$u \to w \to y$	4
z	$u \to w \to z$	3
x	$u \to w \to z \to x$	4

Note: direct link $u \to x$ (cost 5) is **not** in the SPT. The indirect path through w and z is cheaper (cost 4).

From SPT to Forwarding Table

The SPT tells router u: to reach any destination, which **first hop** should I use?

Destination	Next hop	Cost	Path
v	v	2	$u \to v$
w	w	1	$u \to w$
x	w	4	$u \to w \to z \to x$
y	w	4	$u \to w \to y$
z	w	3	$u \to w \to z$

Observation

Four of five destinations use **next hop** w. The SPT structure determines the forwarding table — the router only needs to know the **first hop**, not the full path.

In a real router: "destination" = IP prefix, "next hop" = interface + IP. But the principle is identical.

Every Router Runs Dijkstra

In a link-state network with N routers:

- **Every router** has the same topology database (from flooding)
- Each router runs Dijkstra rooted at **itself**
- Each produces a **different SPT** $\Rightarrow$ different forwarding table

When do routers recompute?

1. A new LSA arrives (link failure, recovery, cost change)
2. An LSA ages out and is refreshed
3. A new router joins the network

In stable networks, Dijkstra runs **infrequently** — the forwarding table stays static.

Convergence

Convergence = all routers agreeing on consistent forwarding tables after a change. Link-state converges fast (seconds) because flooding is fast and each router independently computes a correct result from the complete topology.

OSPF: Open Shortest Path First

OSPF is the most widely deployed link-state routing protocol.

Implements exactly the three-phase process from this lecture:

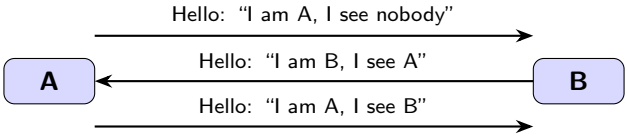
1. **Hello packets** — discover neighbors (default: every 10s on broadcast links)
2. **LSA flooding** — build the link-state database (LSDB) at every router
3. **SPF computation** — Dijkstra on the LSDB $\rightarrow$ forwarding table

Key details:

- Runs directly over IP (protocol number 89 — not TCP, not UDP)
- **Dead interval**: if no hello received for 40s, neighbor declared down $\rightarrow$ new LSA flooded
- RFC 2328 (OSPFv2 for IPv4), RFC 5340 (OSPFv3 for IPv6)
- **IS-IS** is the other major link-state protocol (used by large ISPs)

[Wikipedia: OSPF]

OSPF Hello Protocol



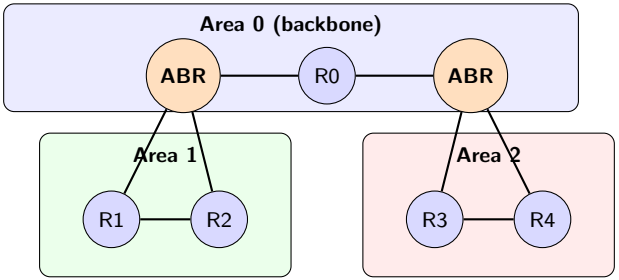
Bidirectional confirmed — adjacency formed, LSA exchange begins

Three purposes:

1. **Neighbor discovery** — find adjacent routers
2. **Bidirectional verification** — confirm the neighbor sees you too
3. **Failure detection** — if hellos stop (dead interval = 40s), declare neighbor down

OSPF Areas

In large networks (hundreds of routers), flooding everywhere is expensive. OSPF uses **areas** to contain flooding and reduce LSDB size.



- **Area 0** = backbone; all other areas must connect to it
- **ABR** (Area Border Router) connects areas and **summarizes** routes between them
- LSA flooding is **contained within an area** — smaller LSDB, faster Dijkstra
- For this course: focus on single-area OSPF (same algorithmic principles)

OSPF Packet Types

OSPF defines five packet types (all carried directly over IP, protocol 89):

Type	Name	Purpose
1	Hello	Neighbor discovery, keep-alive
2	DB Description	Summarize LSDB during initial sync
3	LS Request	Request specific LSAs from neighbor
4	LS Update	Carry one or more LSAs
5	LS Acknowledgment	Confirm receipt (reliable flooding)

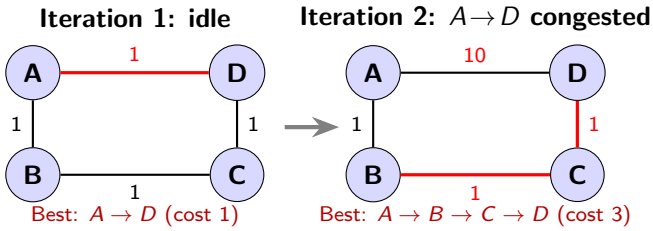
Key fields in an OSPF packet:

- **Router ID**: 32-bit identifier (often highest IP on the router)
- **Area ID**: `0.0.0.0` for backbone
- **LSA age**: counts up from 0; max age = 3600s (1 hour) $\rightarrow$ LSA flushed
- **LS Update**: contains the actual link costs — the raw data for Dijkstra

Note: OSPF packets travel between routers, not end hosts — you won't see them on a laptop capture. Wireshark filter `ospf` works on router captures or in lab environments (GNS3, Mininet).

The Oscillation Problem

What if link costs are based on current traffic load?



Traffic shifts → costs change → path changes → traffic shifts again → **never stabilizes**.

Solutions:

1. **Don't use load-sensitive costs** — OSPF/IS-IS use static costs based on bandwidth. **This is standard practice.**
2. Dampen changes with exponentially weighted moving averages
3. Randomize recomputation times to prevent lock-step oscillation

Modern traffic engineering uses MPLS or segment routing, not OSPF cost changes.

Python: networkx — Dijkstra in 5 Lines

Demo: build the lecture graph and run Dijkstra (see exploration notebook for full code)

```
import networkx as nx

G = nx.Graph()
edges = [("u","v",2), ("u","w",1), ("u","x",5),
         ("v","y",3), ("w","y",3), ("w","z",2),
         ("x","z",1), ("y","z",4)]
for s, d, c in edges:
    G.add_edge(s, d, weight=c)

lengths, paths = nx.single_source_dijkstra(G, "u",
                                           weight="weight")
```

```
u → v:  cost 2, path u → v              u → w:  cost 1, path u → w          u → z:  cost 3, path u
u → x:  cost 4, path u → w → z → x      u → y:  cost 4, path u → w → y
```

The exploration notebook visualizes the SPT, derives the forwarding table, and simulates link failures.

Python: From-Scratch Dijkstra (demo_dijkstra_l15.py)

The core algorithm matching the pseudocode — shared as a standalone script:

```
def dijkstra(G, source):
    G.initialize_source(source) # all D = inf, D(s) = 0
    Q = set(G.V.keys()) # un-finalized nodes
    while Q:
        u = min(Q, key=lambda x: G.V[x].d) # min-cost node
        Q.remove(u) # finalize u
        for v in G.Adj[u]: # check neighbors
            if v in Q:
                relax(u, v, G) # the key step

def relax(u, v, G):
    if G.V[v].d > G.V[u].d + G.w[(u, v)]: # shorter via u?
        G.V[v].d = G.V[u].d + G.w[(u, v)] # update cost
        G.V[v].pi = u # update predecessor
```

Full script: step-by-step trace tables, path reconstruction, forwarding table, link-failure demo.

Shared: demo_dijkstra_l15.py, lecture_15_exploration.py

What's Next

Today completed one family of routing algorithms:

L13: Forwarding tables and longest-prefix match (data plane)

L14: IP addresses and CIDR prefixes (what goes in the table)

L15: Link-state routing and Dijkstra (how the table gets built)

Coming up:

1. **L16 (Thu Mar 26)**: Distance-Vector routing — Bellman-Ford, count-to-infinity, RIP
2. **L17 (Tue Mar 31)**: Autonomous Systems and BGP
3. **L18 (Thu Apr 02)**: IPv4 datagram format, NAT, IPv6, DHCP